COMSATS University Islamabad
Department of Computer Science
CYC386 – Secure Software Design and Development

MedVault: Hardening a Diagnostic Lab Report Portal

Final Report – Midterm Lab Exam: 1-Week DevSecOps Security Sprint (Fall 2026)

Team MedicalVault
Zojaja Naveed (FA23-BCT-047)
Tayyeba Usman (FA23-BCT-053)

Instructor: Engr. Muhammad Ahmad Nawaz
Repository: github.com/Zojaja-Naveed/MedicalVault
November 2026

Abstract*—This report documents a one-week DevSecOps security sprint in which team MedicalVault designed, attacked and hardened MedVault, a diagnostic lab report portal built with Flask, SQLite and a native C report-header parser. Protection needs were elicited from patient, doctor, laboratory, operator and attacker perspectives and mapped to the CIA triad. The system was threat-modelled with context and Level-1 data-flow diagrams, STRIDE, two attack trees and CVSS v3.1 scoring. Five mandatory vulnerability classes — buffer overflow, OS command injection, insecure direct object reference, cross-site request forgery and clickjacking — were deliberately present in a baseline, exploited in a controlled lab, and remediated through reviewed pull requests, together with three additional findings (hard-coded secret, debug mode and missing audit logging). A GitHub Actions pipeline combining exploit-replay tests, AddressSanitizer, Bandit, Semgrep, Trivy and OWASP ZAP failed on the vulnerable baseline (six of eight exploit tests and two High-severity SAST findings) and passed after remediation; branch protection now blocks any merge that fails these gates. Residual risks and lessons learned are discussed.*

***Index Terms*—**DevSecOps, secure software development, threat modeling, STRIDE, CVSS, buffer overflow, command injection, IDOR, CSRF, clickjacking, CI/CD, SAST, DAST.

CONTENTS

I. Introduction
II. System Architecture
III. Development Process and Tooling
IV. Protection Needs Elicitation and CIA Mapping
V. Threat Model Summary
VI. Secure Implementation and Remediation
VII. CI/CD Pipeline and Security Testing Evidence
VIII. CIA Residual Risk Analysis
IX. Lessons Learned
X. Individual Contribution Statement
XI. Conclusion
References
Appendix A. Repository Structure and Deliverables

I. Introduction

Medical laboratory results are among the most sensitive personal data a system can hold. A leaked report discloses a diagnosis; a forged report can drive a wrong treatment. MedVault is a small web portal in which patients receive their laboratory reports as signed `.lab` files, view the results, download compressed copies and grant individual doctors access. It was chosen as the sprint theme because every mandatory vulnerability class arises naturally in its features rather than being bolted on.

The sprint followed the five phases of the CYC386 midterm lab: (1) protection needs elicitation and project setup, (2) threat modeling and quantitative risk, (3) secure implementation, (4) CI/CD and automated security testing, and (5) documentation and submission. Each deliverable explicitly references Confidentiality, Integrity and Availability.

A. Objectives

- Elicit protection needs for all assets and map each one to C, I and/or A, deriving a measurable security requirement.
- Model threats with DFDs, STRIDE, attack trees and CVSS v3.1, and maintain a risk register linked to Jira issues and fix pull requests.
- Demonstrate and then remediate buffer overflow, command injection, IDOR, CSRF and clickjacking, with before/after evidence.
- Build a GitHub Actions pipeline that fails on High/Critical findings, and enforce it through branch protection.

B. Scope and Ethics

All exploitation was performed only against our own local instance (`localhost:5000`) and the GitHub Actions runner. The attacker pages used for CSRF and clickjacking were served from `localhost:8000`. No university or third-party system was targeted. The vulnerable baseline is preserved in the Git history (commit `98478ce`, the state of `main` before PR #6) for reproducibility and is never deployed.

II. System Architecture

MedVault is a server-rendered Flask application. A patient uploads a `.lab` file whose first line has the form `MVLAB|<patient name>|<signature>`. A native C library, `libparser.so`, loaded through `ctypes`, parses this header and reports whether the laboratory signature is present. Report metadata, users and sharing grants are stored in SQLite; report files and generated archives live in a file store. In production the application runs under gunicorn inside a Docker container as an unprivileged user (Fig. 1).

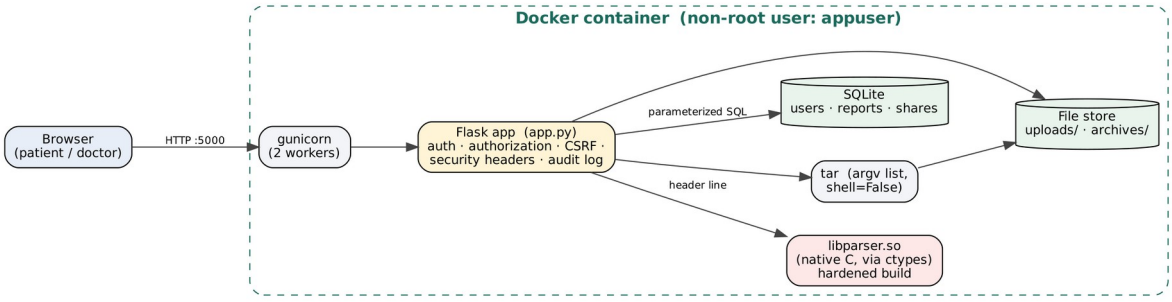


Fig. 1. Deployment architecture of the hardened MedVault container.

TABLE I
Components and Technologies

Component	Technology	Security-relevant role
Web application	Python 3.12, Flask 3.1, Jinja2	Authentication, central authorization, CSRF tokens, security headers, audit log
Native parser	C (gcc), `libparser.so` via `ctypes`	Parses untrusted report headers; the only memory-unsafe module, kept small and isolated
Database	SQLite, parameterized queries	Users (salted password hashes), reports, sharing grants
File store	`uploads/`, `archives/`	Report files and `.tar.gz` downloads

Component	Technology	Security-relevant role
Application server	gunicorn 23 in Docker (`python:3.12-slim`)	No debug server; runs as non-root `appuser`
CI/CD	GitHub Actions	Build, exploit-replay tests, ASAN, Bandit, Semgrep, Trivy, OWASP ZAP
Process	GitHub (protected `main`), Jira Scrum	Peer-reviewed pull requests, traceability from PNE to fix

A. User Roles and Features

Patients (`alice`, `bob`) see their own reports, upload new ones, give a doctor access and create compressed downloads. Doctors (`drkhan`, and the attacker-controlled `drevil` used in demonstrations) see only reports shared with them. Each report page renders the laboratory sheet with reference ranges and High/Low flags, and a stamp showing whether the lab signature was verified (Fig. 2, Fig. 3).

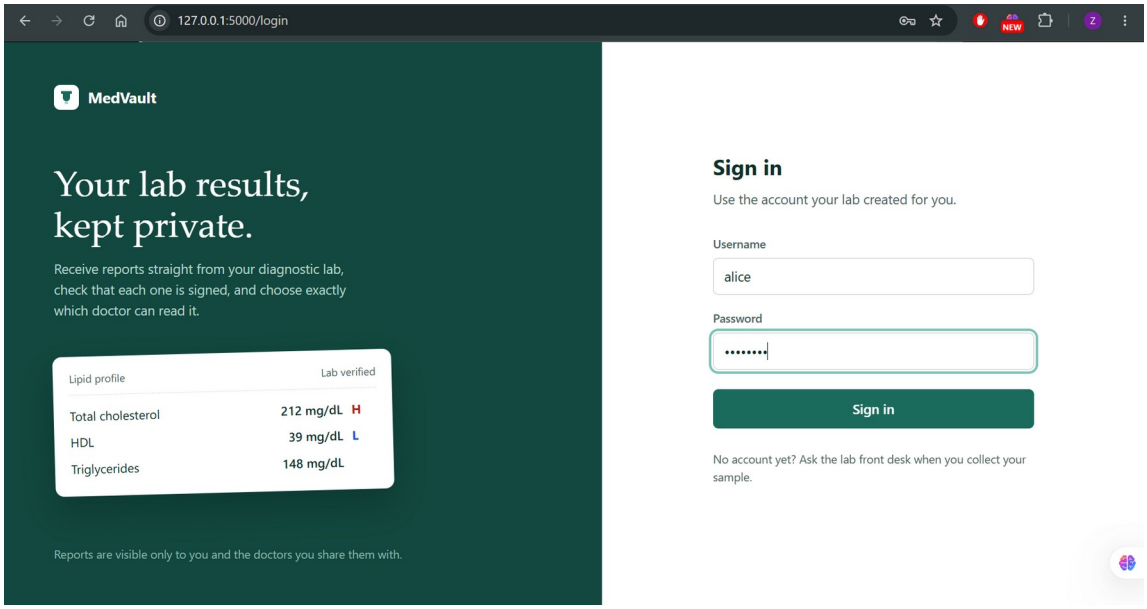


Fig. 2. MedVault sign-in page (no credentials are displayed).

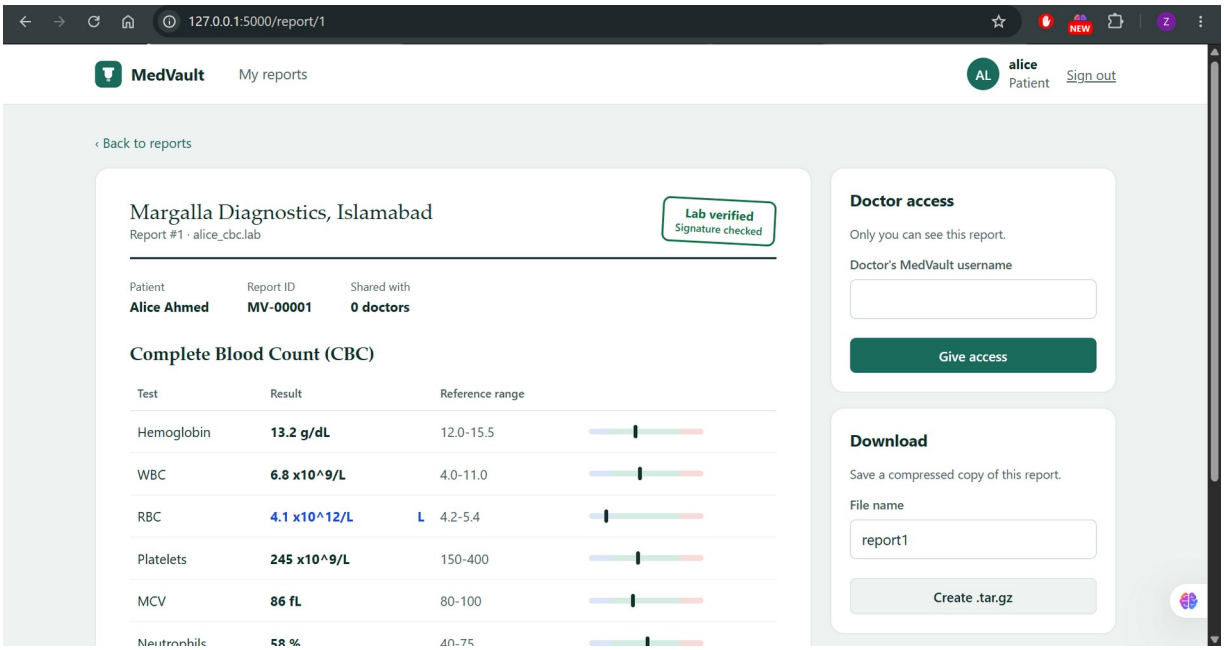


Fig. 3. Report view for patient Alice: lab letterhead, verification stamp, flagged results, doctor access and download panels.

III. Development Process and Tooling

A. Git Workflow and Branch Protection

The repository `MedicalVault` uses a protected `main` branch. Direct pushes are disabled; every change lands through a pull request from a `feature/*`, `fix/*` or `docs/*` branch, and each pull request requires one approving review from the other team member. After the pipeline was introduced, the three CI jobs were added as required status checks with "require branches to be up to date", so a red pipeline physically blocks merging (Fig. 4, Fig. 5).

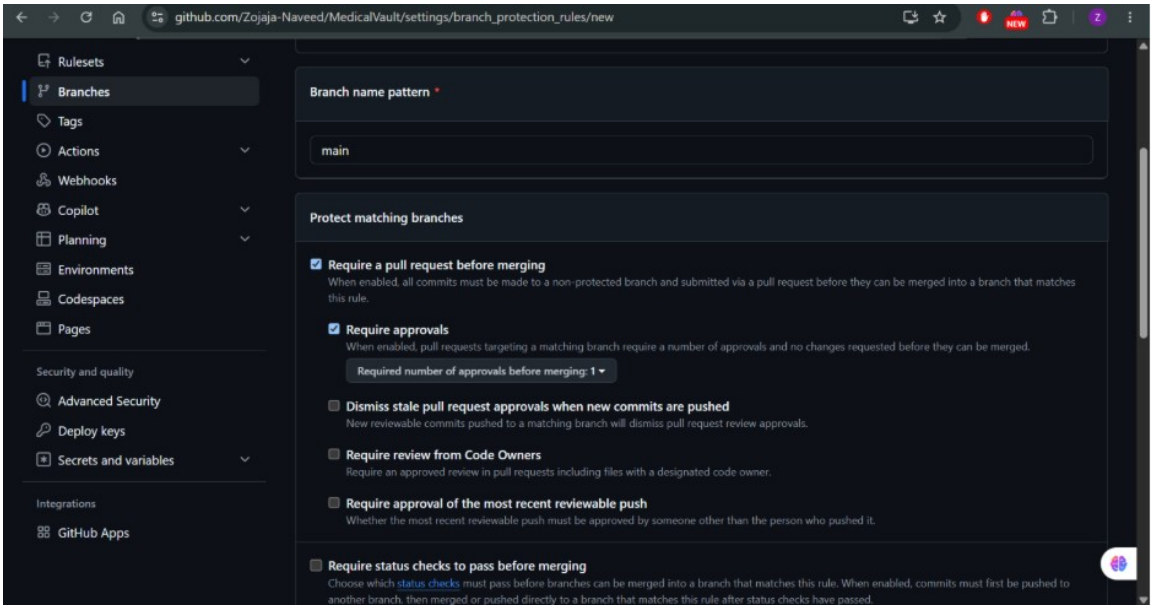


Fig. 4. Initial protection rule for main: pull request and one approval required.

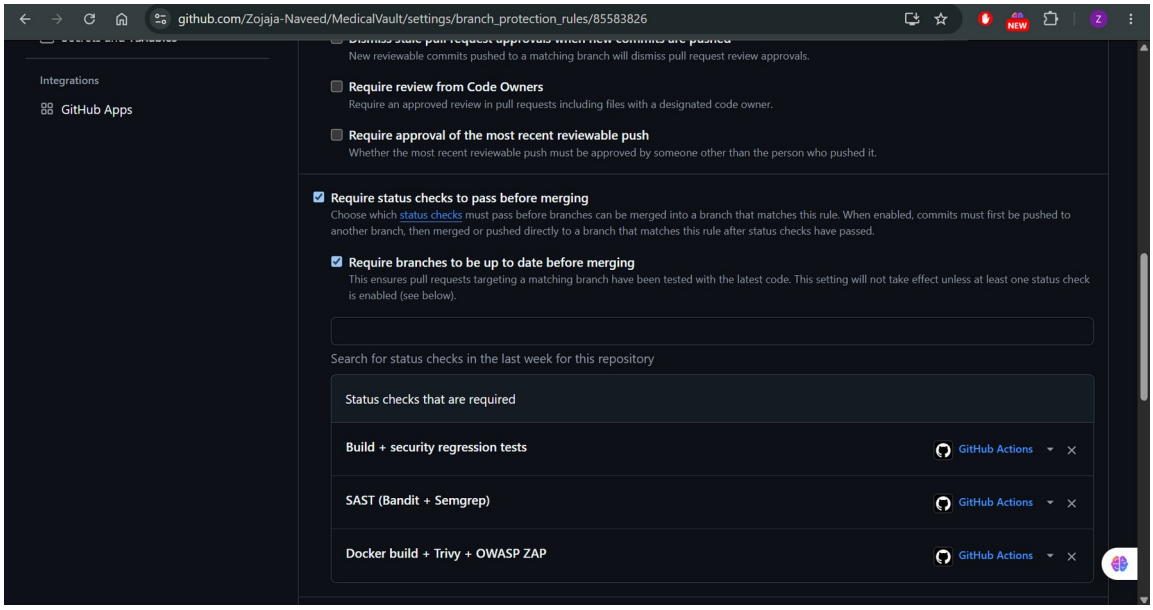


Fig. 5. Final rule: the three security jobs are required status checks.

TABLE II
Pull Requests Merged During the Sprint

PR	Branch	Purpose	Jira	Review
#1	docs/add-member-Zojaja	Workflow test: add member to README	MV-8	Approved by Tayyeba Usman
#2	docs/add-member-tayyebacodes	Workflow test: second member's commit	MV-8	Reviewed and merged
#3	docs/pne-threat-model	PNE report, threat model, DFDs, attack trees	MV-6 – MV-12	Approved and merged by Tayyeba Usman
#4	feature/new-ui	Realistic lab-report UI, samples, attacker pages	MV-13	Approved and merged by Tayyeba Usman

PR	Branch	Purpose	Jira	Review
#5	feature/ci-cd-pipeline	CI/CD security pipeline and exploit-replay tests	MV-19	Approved; red run recorded as baseline
#6	fix/buffer-overflow	Bounds check in native parser	MV-14	Approved
#7	fix/web-vulnerabilities	Fix command injection, IDOR, CSRF, clickjacking, secret, debug, audit log; Trivy/ZAP stage	MV-15 – MV-18, MV-20	Approved

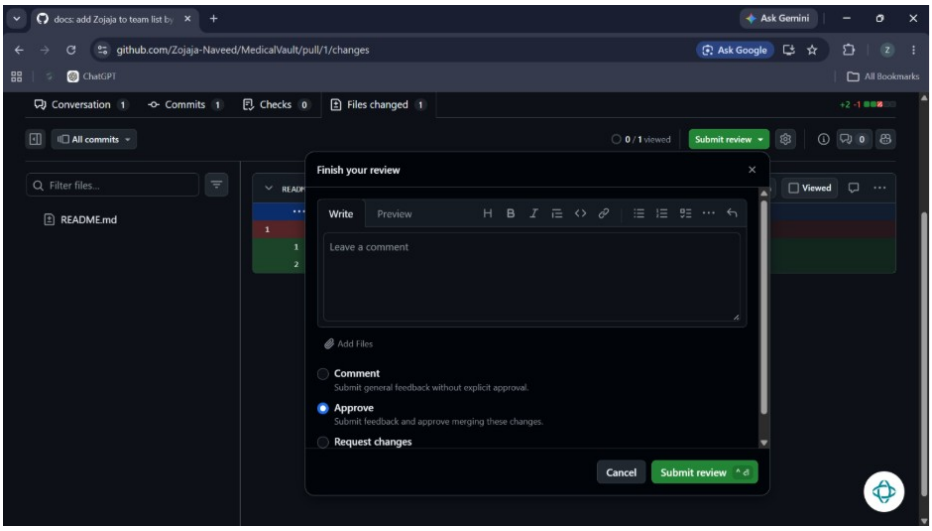


Fig. 6. Peer review: the reviewer approves a pull request before it can be merged.

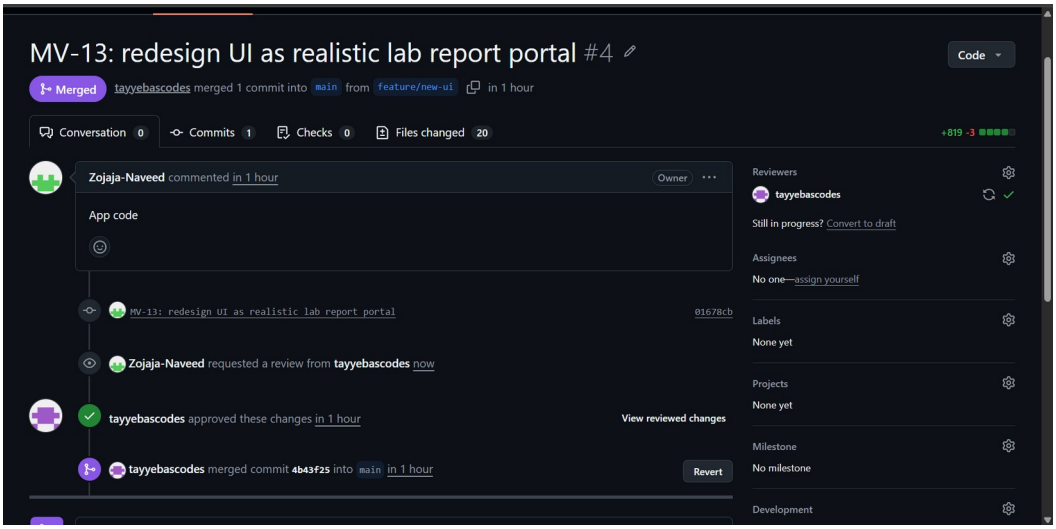


Fig. 7. PR #4 reviewed, approved and merged by the second team member.

B. Jira Scrum Board and Traceability

A Jira Scrum project, MedVault Security Sprint (key MV), holds five epics — PNE & CIA Mapping, Threat Modeling & Risk, Secure Implementation, CI/CD & Security Testing, and Documentation & Demo — with nineteen tasks beneath them. Every vulnerability task names its protection need and threat (e.g., MV-16 addresses PN-01 and T-03). Jira keys appear in every commit message and pull request title, and each issue carries a comment linking the merged pull request, giving the chain PNE → threat → Jira issue → pull request → test evidence.

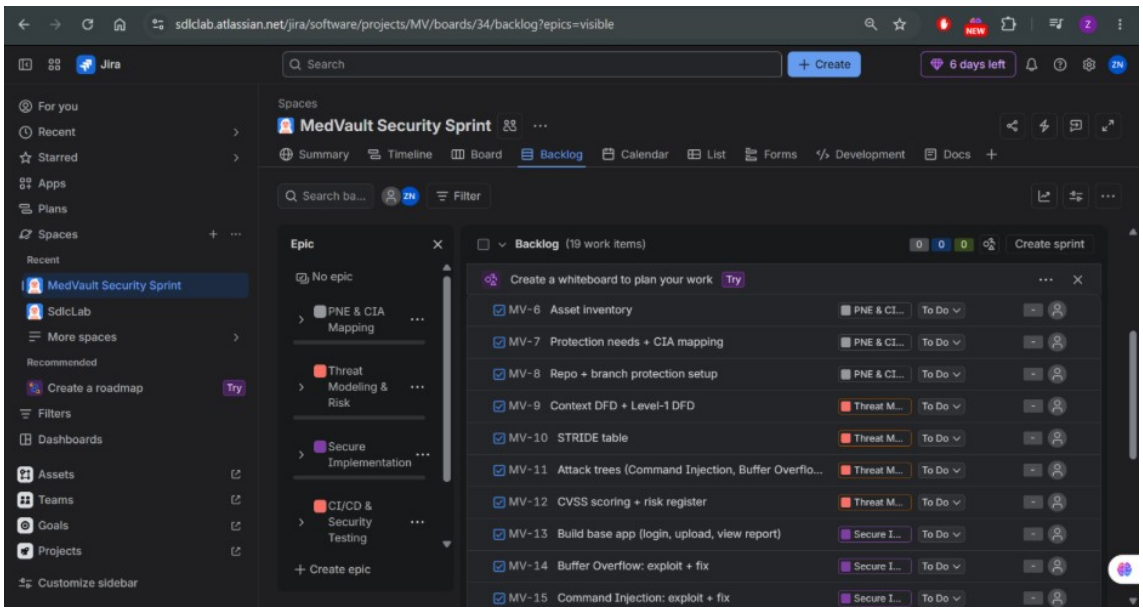


Fig. 8. Sprint backlog at the start of the sprint with the five epics.

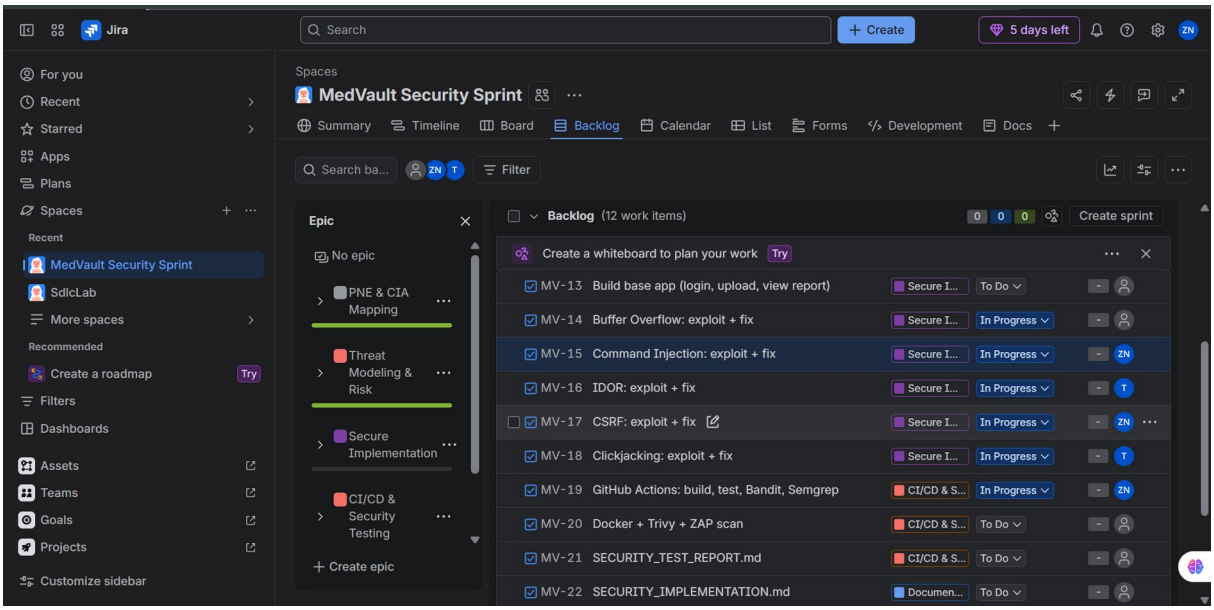


Fig. 9. Mid-sprint: vulnerability tasks in progress after exploitation was confirmed.

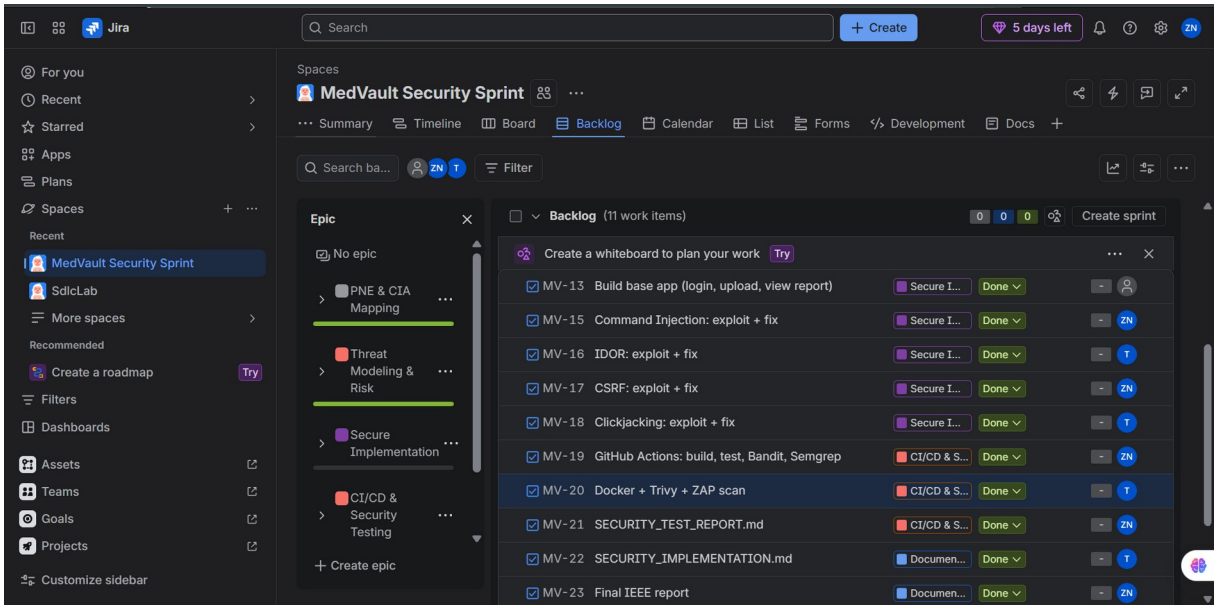


Fig. 10. Board at sprint closure with completed tasks.

IV. Protection Needs Elicitation and CIA Mapping

Assets were inventoried across data, components, processes and external entities (Table III). Needs were elicited from the patient, doctor, laboratory and operator perspectives and from misuse cases written from the attacker's perspective. Each need is mapped to the CIA triad and turned into a measurable security requirement (Table V); the full analysis is in `PNE_Report.pdf`.

TABLE III
Asset Inventory

ID	Asset	Type	Description
A1	Lab report files	Data	Uploaded `.lab` medical reports
A2	Report metadata	Data	Owner, patient name, verified flag, file path
A3	User credentials	Data	Usernames and salted password hashes
A4	Session cookies	Data	Signed Flask session
A5	Share permissions	Data	Which doctor may read which report
A6	Flask application	Component	Login, upload, view, share, compress
A7	Native C parser	Component	Parses the untrusted report header
A8	SQLite database	Component	Stores A2, A3, A5
A9	File store	Component	Stores A1 and archives
A10	Audit log	Data	Security-relevant user actions

TABLE IV
Misuse Cases

ID	Misuse case (attacker perspective)
MU-1	Read another patient's report by changing the report ID in the URL
MU-2	Upload a crafted report whose long name forges the "verified" stamp or crashes the parser
MU-3	Inject shell commands through the archive name
MU-4	Make a logged-in patient share a report with the attacker from another website
MU-5	Frame MedVault invisibly and trick the patient into clicking "Give access"
MU-6	Forge sessions using a hard-coded secret or abuse debug mode

TABLE V
Protection Needs, CIA Mapping and Security Requirements

ID	Protection need	C	I	A	Measurable security requirement	Threat	Jira
PN-01	Only the owner and doctors explicitly granted access can read a report	✔			Object-level authorization on every report access; unauthorized IDs return 404	T-03	MV-16
PN-02	A report's verified stamp cannot be forged		✔		Parser rejects names ≥ 32 bytes; bounded copies; hardened build	T-01	MV-14
PN-03	The server never executes user-supplied commands	✔	✔	✔	No shell interpretation of user input; archive names match `^[A-Za-z0-9_-]{1,40}$`	T-02	MV-15
PN-04	State-changing actions happen only when the user intends them	✔	✔		Valid per-session CSRF token on every POST; `SameSite=Strict`	T-04	MV-17
PN-05	Users cannot be tricked into clicking hidden UI		✔		`X-Frame-Options: DENY` and CSP `frame-ancestors 'none'` on all responses	T-05	MV-18
PN-06	A malformed upload cannot crash the service			✔	Malformed headers rejected; 1 MB upload limit	T-01	MV-14
PN-07	Credentials and sessions are protected	✔	✔		Hashed passwords; secret from environment; `HttpOnly`; debug off	T-06, T-07	MV-13
PN-08	Security-relevant actions are attributable		✔		Audit log of login, upload, share, compress and denials	T-08	MV-13

Confidentiality is covered by PN-01, PN-03, PN-04 and PN-07; Integrity by PN-02 to PN-05, PN-07 and PN-08; Availability by PN-03 and PN-06. Integrity has the widest coverage because a forged or tampered medical report is as harmful as a leaked one.

V. Threat Model Summary

The full model is in `THREAT_MODEL.pdf`. The context diagram (Fig. 11) shows MedVault as one process exchanging data with patients and doctors; the laboratory hands signed files to patients outside the system. The Level-1 DFD (Fig. 12) decomposes it into five processes, the native parser and three data stores. The trust boundary encloses the server: every flow that crosses it carries untrusted input and must be validated and authorized.

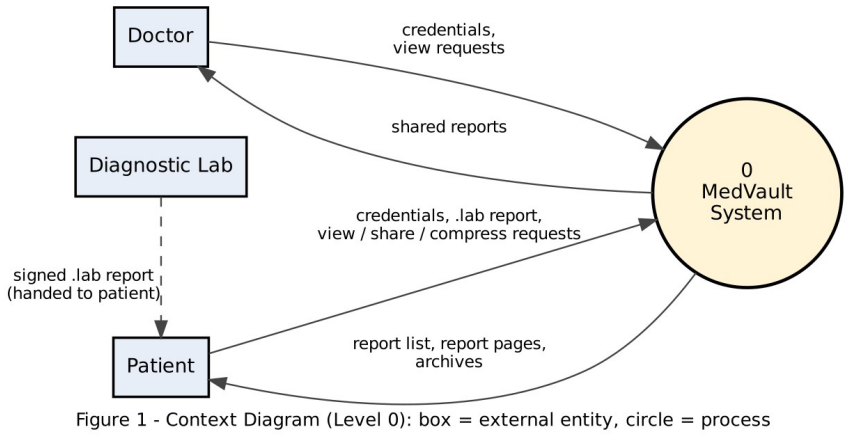


Fig. 11. Context diagram (Level 0).

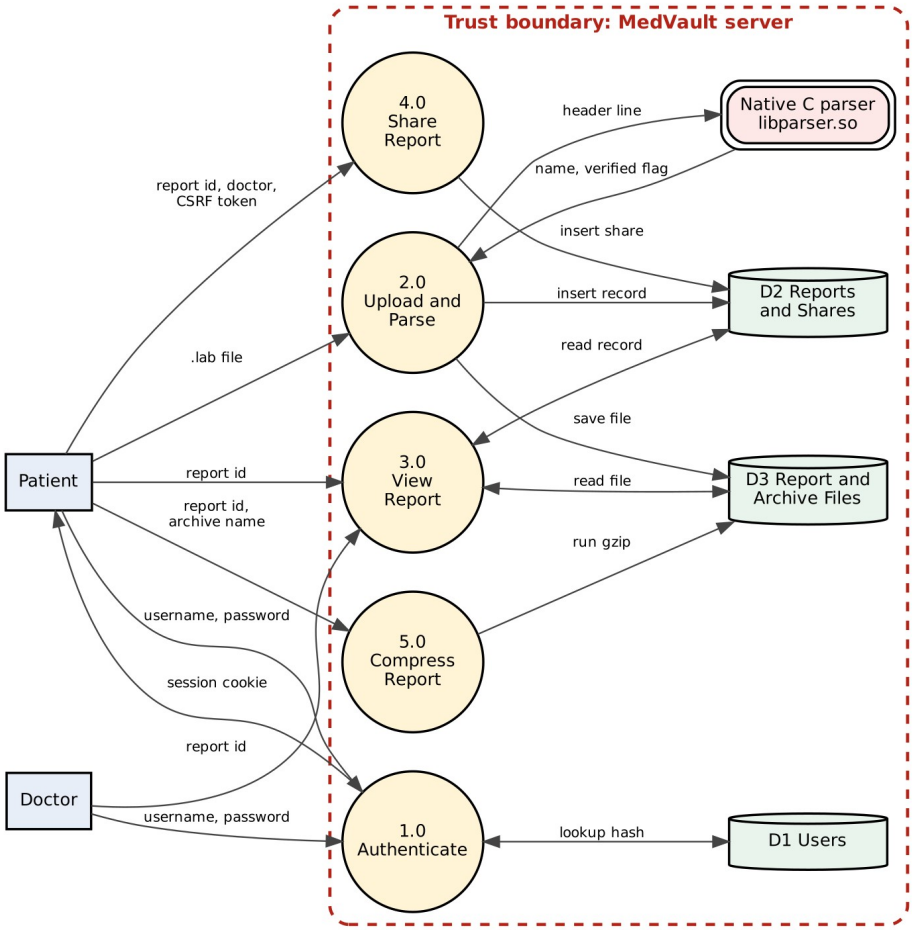


Fig. 12. Level-1 data-flow diagram with the server trust boundary.

A. STRIDE Analysis

STRIDE was applied to every DFD element. Table VI summarizes the eight threats derived from it; the per-element matrix is in the threat model.

TABLE VI
THREATS DERIVED FROM STRIDE

ID	Threat	STRIDE	DFD element	PN
T-01	Buffer overflow in native report-header parser	T, D, E	P2 Upload & Parse, native parser	PN-02, PN-06
T-02	OS command injection in Compress Report	T, I, D, E	P5 Compress, D3 file store	PN-03
T-03	IDOR on `/report/<id>`	I, E	P3 View Report, D2	PN-01
T-04	CSRF on Share Report	S, T, I	P4 Share, browser data flow	PN-04
T-05	Clickjacking of report / share UI	S, I	P4 Share	PN-05
T-06	Hard-coded Flask secret → session forgery	S	P1 Authenticate	PN-07
T-07	Flask debug mode enabled	I, E	P1 / whole app	PN-07
T-08	Missing audit logging	R	All processes	PN-08

B. Attack Trees

Attack Tree 1 (Fig. 13) has the goal of executing OS commands on the server. Its easiest path — log in with any account, put shell metacharacters in the archive name, server passes the string to a shell — needs only a free account and one form field, which is why T-02 is ranked first. Attack Tree 2 (Fig. 14) has the goal of presenting an unsigned report as lab-verified; its first branch is the overflow demonstrated in Section VI-A, and its third branch (copying a static signature marker) remains a residual risk.

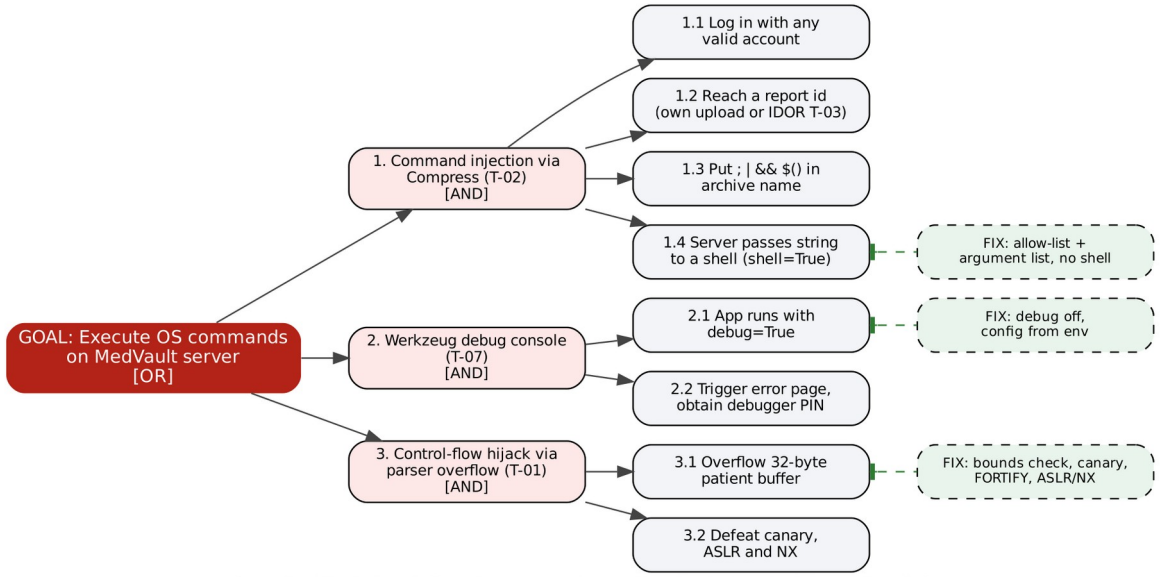


Figure 3 - Attack Tree 1: Remote command execution. Easiest path: 1.1 -> 1.3 -> 1.4

Fig. 13. Attack tree 1: remote command execution, with mitigations attached to the leaves they block.

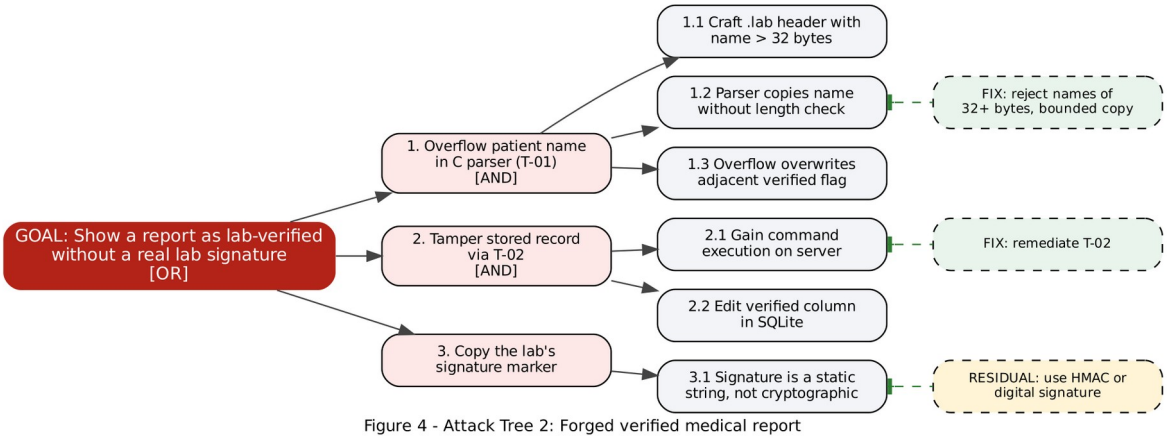


Fig. 14. Attack tree 2: forged "verified" medical report.

C. CVSS v3.1 Scoring and Risk Register

TABLE VII
CVSS v3.1 Base Scores

ID	Threat	Vector	Score	Justification
T-02	Command injection	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H	8.8 High	Any account, no interaction, full control of the server
T-01	Buffer overflow	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:H	8.1 High	Forges verified flag or crashes service; no data read
T-07	Debug mode	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H	8.1 High	Debugger console executes code; PIN raises complexity
T-06	Hard-coded secret	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N	7.4 High	Session forgery once the key is found
T-04	CSRF	AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:L/A:N	7.1 High	Victim only visits a page; report disclosed to attacker
T-03	IDOR	AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N	6.5 Medium	Any user reads any report; 8.3 with CR:H
T-05	Clickjacking	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	4.3 Medium	Requires a victim click; limited integrity impact

Because medical data demands high confidentiality, the environmental requirement `CR:H` raises IDOR from 6.5 to 8.3, so it was treated as High priority. The prioritized register (Table VIII) links every threat to its Jira issue and remediation pull request.

TABLE VIII
Prioritized Risk Register

Priority	ID	Threat	CVSS	Jira	Fix PR	Status	Residual
1	T-02	Command injection	8.8	MV-15	#7	Fixed	Low
2	T-03	IDOR	6.5 / 8.3	MV-16	#7	Fixed	Low
3	T-01	Buffer overflow	8.1	MV-14	#6	Fixed	Low
4	T-07	Debug mode	8.1	MV-13	#7	Fixed	Low
5	T-06	Hard-coded secret	7.4	MV-13	#7	Fixed	Low
6	T-04	CSRF	7.1	MV-17	#7	Fixed	Low
7	T-05	Clickjacking	4.3	MV-18	#7	Fixed	Very low
8	T-08	No audit log	—	MV-13	#7	Fixed	Low

VI. Secure Implementation and Remediation

For each class this section gives the vulnerable code, the lab exploit and its evidence, the fix, and the verification. The vulnerable baseline was built and run with Docker (`medvault:vulnerable`); the hardened image was rebuilt from `main` after the fixes were merged. Every fix is also locked in by an exploit-replay regression test (Section VII).

A. Buffer Overflow (T-01, CWE-120)

Vulnerability:

The parser copies the patient name into a 32-byte stack buffer without comparing its length, and the `verified` flag is stored directly after the buffer:

```
typedef struct { char patient[32]; int verified; } ReportHeader;
h.verified = (end && strncmp(end + 1, "LABSIGNED", 9) == 0) ? 1 : 0;
memcpy(h.patient, p, n);   /* n is never compared with 32 */
h.patient[n] = '\0';
```

Exploit:

A report with a 36-character name and the signature `NOT-SIGNED` (Fig. 15) overwrites `verified` with non-zero bytes, so the unsigned report is displayed as **Lab verified**, while an honest unsigned report correctly shows Unverified (Fig. 16). The command-line driver confirms it: `result=0` for a normal unsigned header, `result=1` for the overflow (Fig. 17). A 300-character name smashes the stack and kills the server process — a denial of service for every user (Fig. 18).

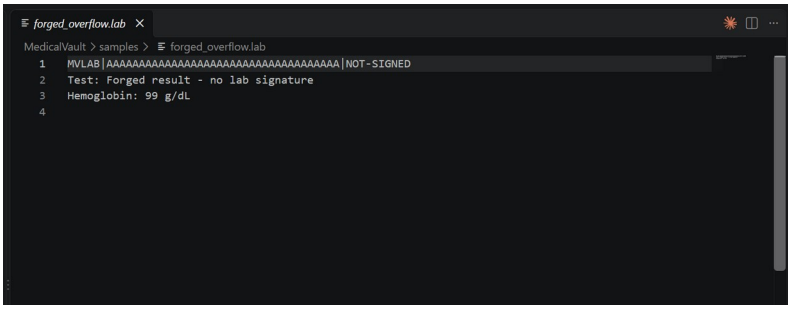


Fig. 15. Overflow payload: 36-byte patient name with an invalid signature.

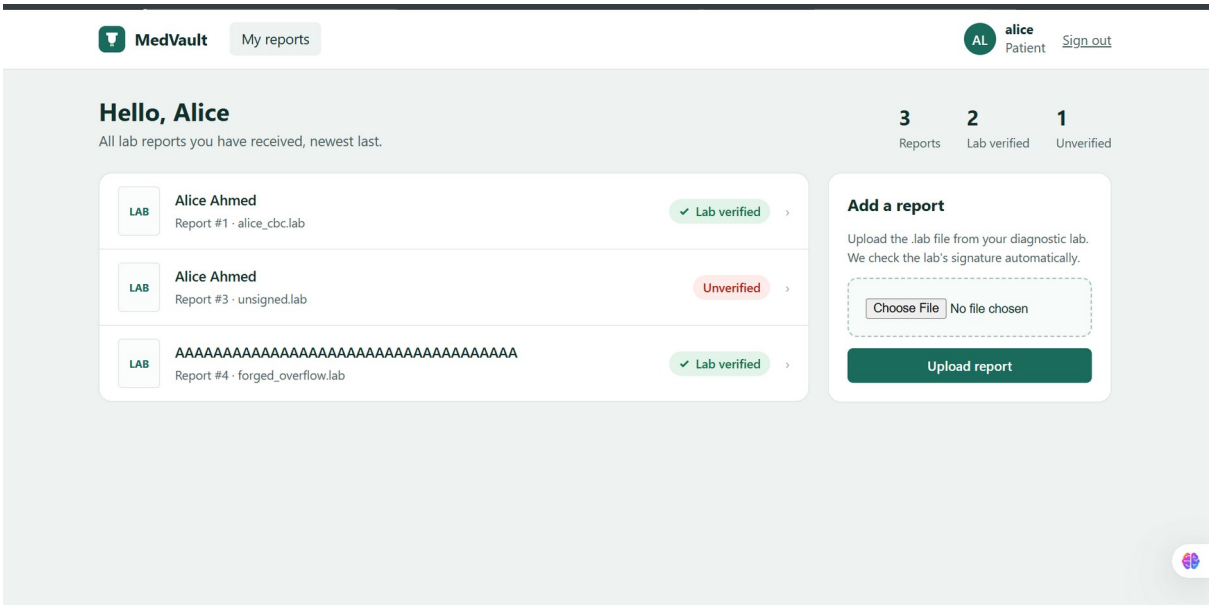


Fig. 16. Data corruption: the unsigned overflow report is shown as "Lab verified".

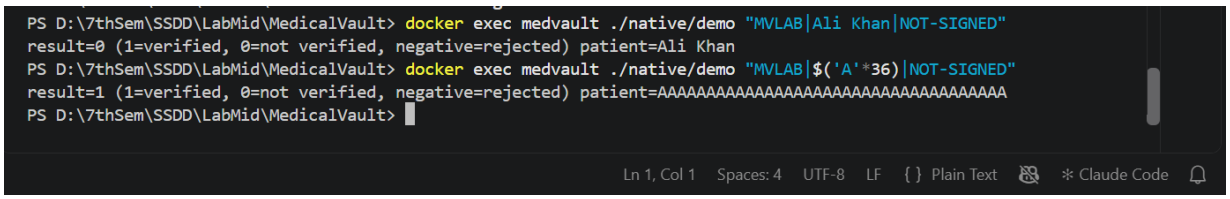


Fig. 17. Parser driver: the 36-byte name returns result=1 (verified).

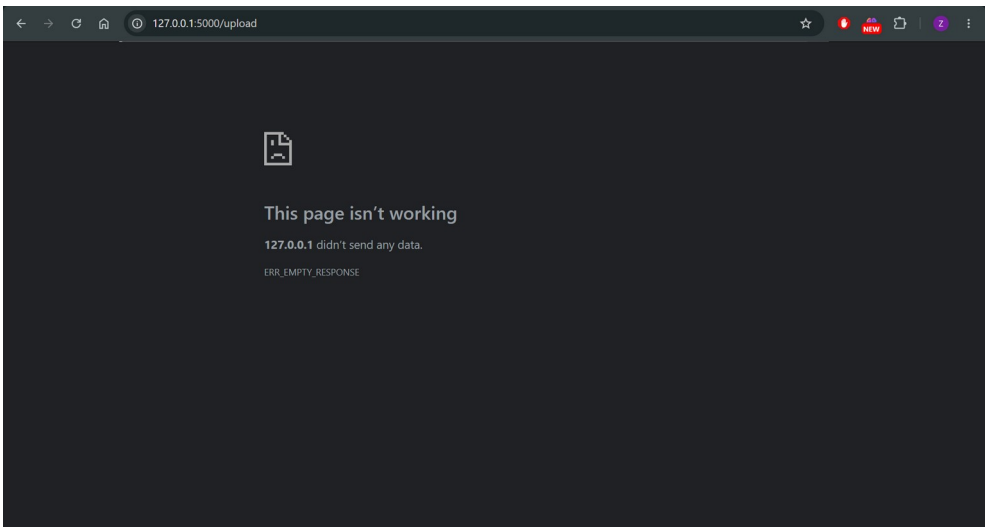


Fig. 18. Denial of service: a 300-byte name crashes the server (ERR*EMPTY*RESPONSE).

Fix:

```
if (strnlen(input, MAX_INPUT) >= MAX_INPUT) return -2;   /* overall limit          */
if (n == 0 || n >= sizeof h.patient) return -2;           /* reject oversize name   */
memcpy(h.patient, p, n);                                  /* safe: n < 32           */
h.patient[n] = '\0';
snprintf(out, out_len, "%s", h.patient);                  /* bounded copy out       */
```

The client-supplied length is never trusted. The library is compiled with `-fstack-protector-strong` (stack canary), `-D_FORTIFY_SOURCE=2` (checked libc calls) and `-Wl,-z,relro,-z,now` (full RELRO); ASLR and NX are enabled by the platform. AddressSanitizer runs in CI.

Verification:

The forged file is now rejected with "Invalid report header" and the server stays up (Fig. 19); the driver returns `result=-2` (Fig. 20). In the pipeline the ASAN step, which reported `stack-buffer-overflow` before the fix, passes (Fig. 21).

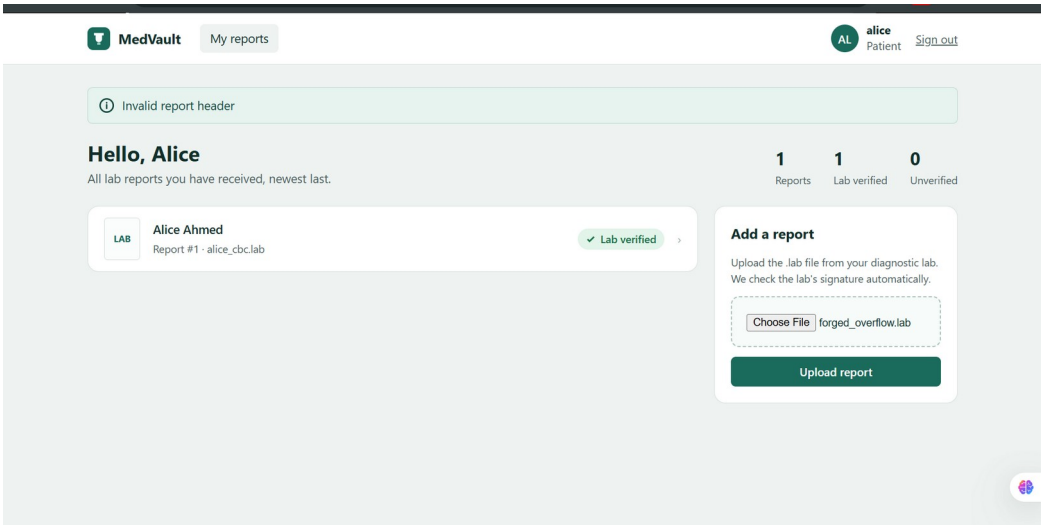


Fig. 19. After PR #6: overflow upload rejected, no forged row, service available.

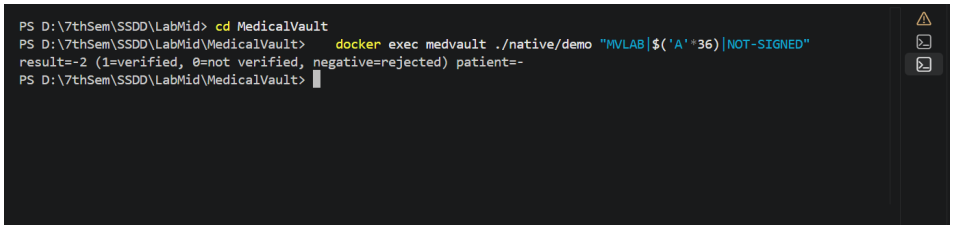


Fig. 20. After PR #6: the parser rejects the 36-byte name (result=-2).

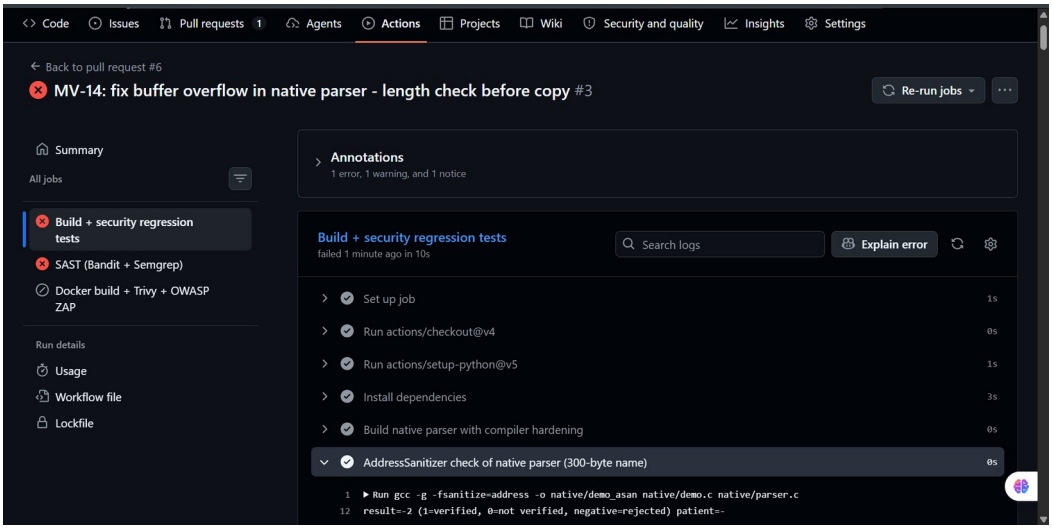


Fig. 21. Pipeline on PR #6: AddressSanitizer check now passes.

B. OS Command Injection (T-02, CWE-78)

Vulnerability:

```
cmd = f"tar -czf archives/{archive}.tar.gz -C uploads {report['stored_name']}"
subprocess.run(cmd, shell=True, cwd=BASE)
```

Exploit:

The archive name `x; id > static/leak.txt #` ends the `tar` command with `;`, runs `id` and comments out the rest with `#` (Fig. 22). Requesting `/static/leak.txt` returns `uid=0(root)` — the injected command ran as root (Fig. 23).

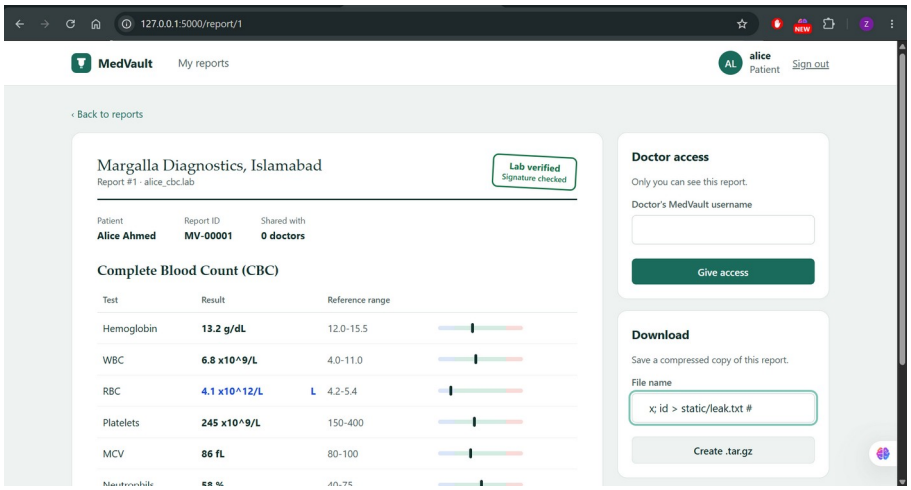


Fig. 22. Injection payload entered in the download file-name field.

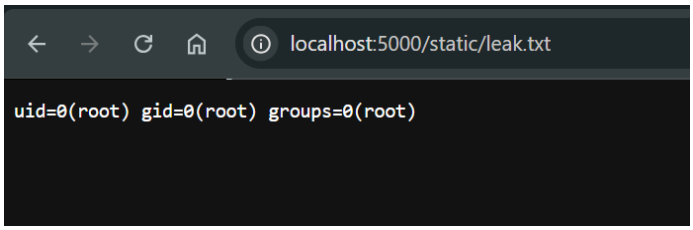


Fig. 23. Injected command output: executed on the server as root.

Fix:

```
ARCHIVE_NAME_RE = re.compile(r"^[A-Za-z0-9_-]{1,40}$")
if not ARCHIVE_NAME_RE.fullmatch(archive):
    abort(400, "Archive name may contain only letters, digits, - and _")
subprocess.run(["tar", "-czf", os.path.join(ARCHIVES, archive + ".tar.gz"),
                "-C", UPLOADS, report["stored_name"]],
               shell=False, check=True, timeout=15)
```

Two independent layers: an allow-list rejects every shell metacharacter (`; | & $ ( )` and backticks), and `tar` is executed with an argument vector and `shell=False`, so no shell interprets the input (execve-style). The container also runs as a non-root user.

Verification:

The same payload now returns **400 Bad Request** and no file is created (Fig. 24); Bandit no longer reports B602.

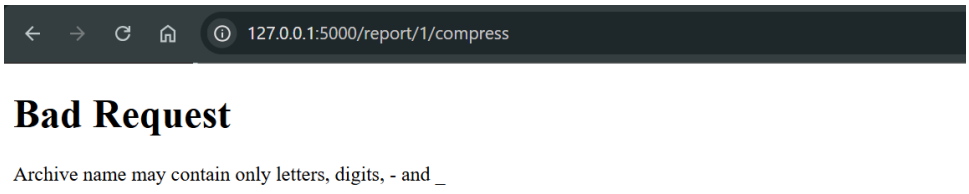


Fig. 24. After PR #7: the payload is rejected with 400 Bad Request.

C. Insecure Direct Object Reference (T-03, CWE-639)

Vulnerability and exploit:

`view_report` loaded a report by ID only. Logged in as alice, changing `/report/1` to `/report/2` displayed Bob's confidential lipid profile (Fig. 25).

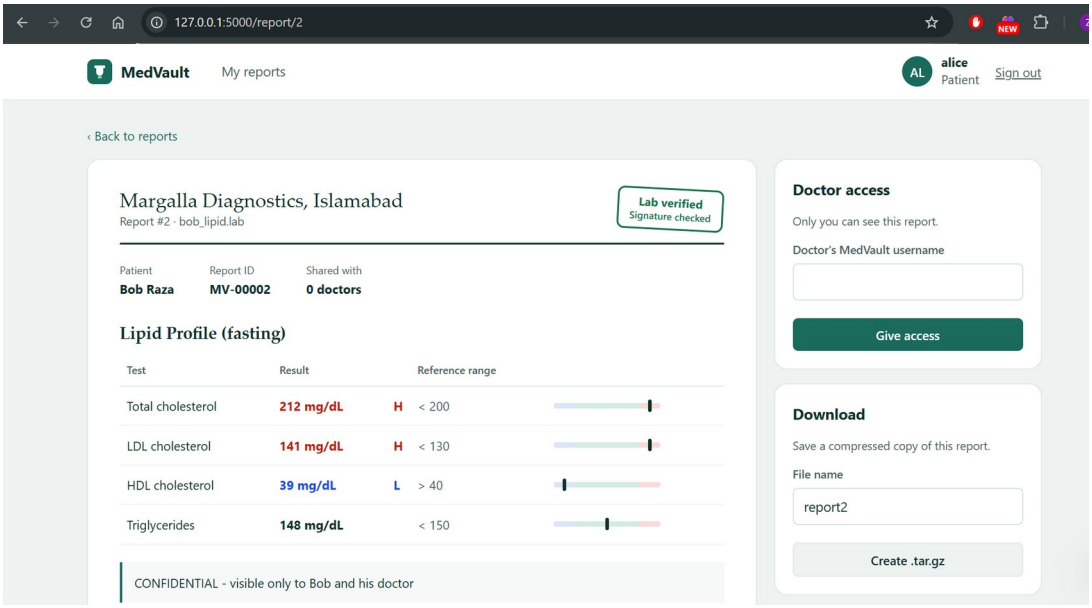


Fig. 25. IDOR: alice reads Bob's confidential report by changing the ID.

Fix:

A single authorization function is used by every report route, following the course rule to centralize authorization rather than scatter checks:

```
def get_report_for_user(report_id, owner_only=False):
    report = db().execute("SELECT * FROM reports WHERE id = ?", (report_id,)).fetchone()
    if report is None or user is None:            abort(404)
    if report["owner_id"] == user["id"]:          return report
    if not owner_only and user["role"] == "doctor" and <shared with this doctor>:
        return report
    audit.info("DENIED user=%s report=%s", user["username"], report_id)
    abort(404)
```

A 404 rather than 403 is returned so that an attacker cannot enumerate which report IDs exist. Sharing and compressing are owner-only.

Verification:

`/report/2` as alice now returns 404 (Fig. 26); a second test confirms the owner can still open her own report.

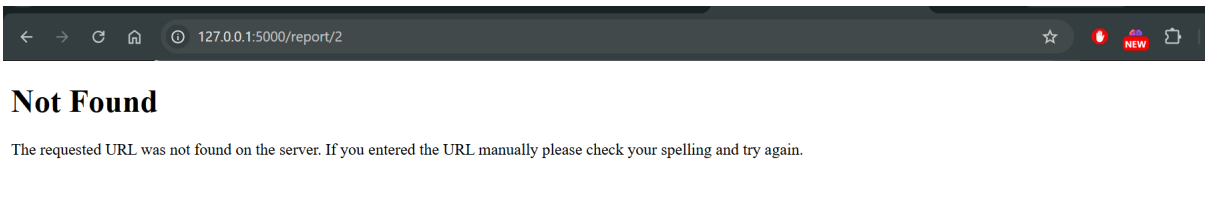


Fig. 26. After PR #7: another patient's report returns 404 Not Found.

D. Cross-Site Request Forgery (T-04, CWE-352)

Vulnerability and exploit:

The share form carried no anti-CSRF token. While alice was logged in, opening the attacker page `csrf.html` silently posted `doctor=drevil` to `/report/1/share`; MedVault confirmed "Report shared with drevil" (Fig. 27).

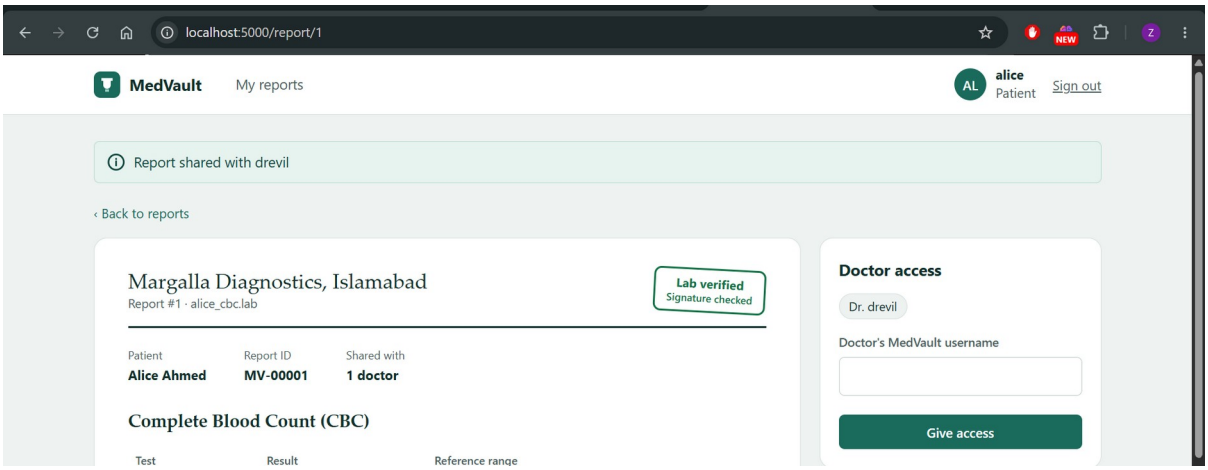


Fig. 27. CSRF: an attacker page grants the attacker's doctor account access to report 1.

Fix:

```
def csrf_token():                       # random 256-bit token per session
    if "csrf_token" not in session:
        session["csrf_token"] = secrets.token_urlsafe(32)
    return session["csrf_token"]

@app.before_request
def check_csrf():                       # enforced on every POST
    if request.method == "POST":
        sent, expected = request.form.get("csrf_token", ""), session.get("csrf_token", "")
        if not expected or not hmac.compare_digest(sent, expected):
            abort(400, "Invalid or missing CSRF token")

app.config["SESSION_COOKIE_SAMESITE"] = "Strict"
app.config["SESSION_COOKIE_HTTPONLY"] = True
```

This is the synchronizer-token pattern with a constant-time comparison, plus `SameSite=Strict` cookies as defence in depth.

Verification:

The same attacker page now receives **400 – Invalid or missing CSRF token** and the share list is unchanged (Fig. 28).

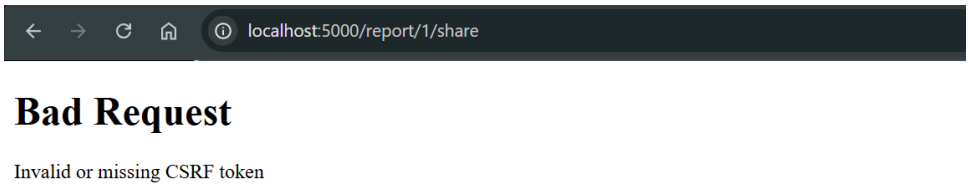


Fig. 28. After PR #7: the forged request is rejected.

E. Clickjacking (T-05, CWE-1021)

Vulnerability and exploit:

No framing restrictions were sent. `clickjack.html` loads report 1 in a transparent iframe aligned over a fake promo-code box and a CLAIM PRIZE button; the victim types `drevil` and clicks, actually granting access (Fig. 29; opacity 30 % for visibility).

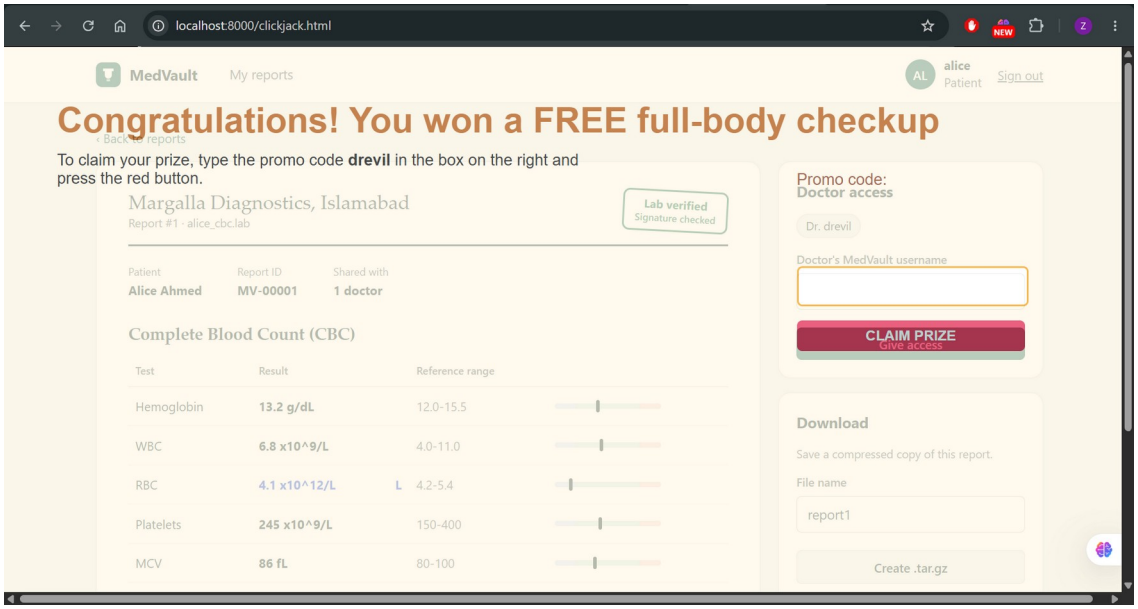


Fig. 29. Clickjacking: MedVault framed under the attacker's fake button.

Fix:

```
@app.after_request
def security_headers(resp):
    resp.headers["X-Frame-Options"] = "DENY"
    resp.headers["Content-Security-Policy"] = "frame-ancestors 'none'; default-src 'self';
style-src 'self' 'unsafe-inline'"
    resp.headers["X-Content-Type-Options"] = "nosniff"
    resp.headers["Referrer-Policy"] = "no-referrer"
    return resp
```

Verification:

The browser refuses to render MedVault inside the frame ("localhost refused to connect"), leaving nothing to click (Fig. 30).

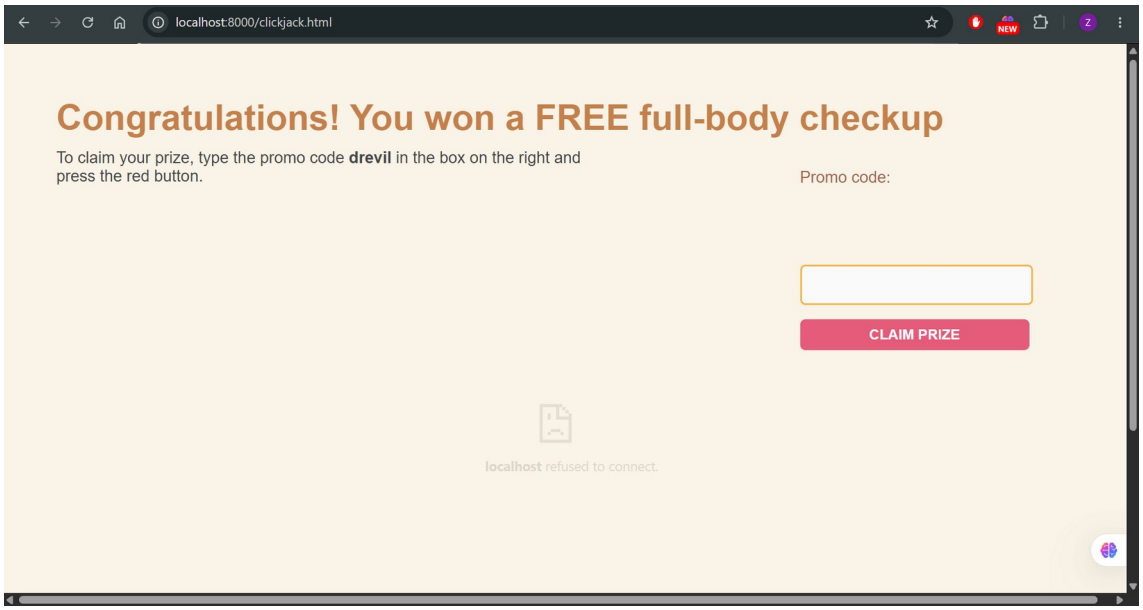


Fig. 30. After PR #7: framing is blocked by the browser.

F. Additional Findings

TABLE IX
ADDITIONAL VULNERABILITIES FIXED

ID	Finding	Source	Fix	CIA
T-06	Hard-coded `SECRET_KEY` allows session forgery	Code review	Key from `MEDVAULT_SECRET_KEY` env var, random fallback	C, I
T-07	`app.run(debug=True)` exposes Werkzeug debugger	Bandit B201 (High)	`debug=False`; gunicorn; non-root container	C, I, A
T-08	No audit trail (Repudiation)	STRIDE	Audit log for login, upload, share, compress, denials, CSRF rejections	I
PN-06	Unbounded uploads	PNE	`MAX_CONTENT_LENGTH` = 1 MB	A

VII. CI/CD PIPELINE AND SECURITY TESTING EVIDENCE

The workflow `.github/workflows/ci-cd.yml` runs on every push and pull request to `main` and `develop` (Fig. 31). Job 1 compiles the parser with hardening flags, runs it under AddressSanitizer with a 300-byte name and executes eight exploit-replay tests. Job 2 runs Semgrep (`p/python`, `p/flask`) and Bandit, failing on any High finding. Job 3 runs only if both pass: it builds the Docker image, scans it with Trivy (failing on fixable Critical CVEs), starts it and runs an OWASP ZAP baseline scan. Test, SAST and ZAP reports are retained as run artifacts.

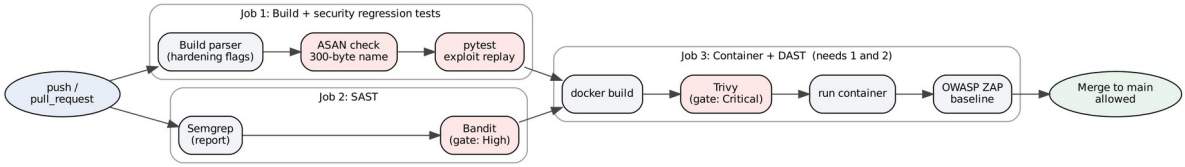


Fig. 31. Pipeline stages and security gates (red = build-failing gate).

TABLE X
EXPLOIT-REPLAY REGRESSION TESTS

Test	Threat	Assertion
`test_idor_other_patients_report_is_blocked`	T-03	alice → `/report/2` returns 404
`test_owner_can_still_view_own_report`	—	no over-blocking
`test_command_injection_is_rejected`	T-02	injected file is never created
`test_csrf_request_without_token_is_rejected`	T-04	400 and no share
`test_clickjacking_headers_present`	T-05	XFO DENY and frame-ancestors 'none'
`test_overflow_cannot_forge_verified_flag`	T-01	36-byte name not verified
`test_valid_signed_header_still_verified`	—	genuine report accepted
`test_no_hardcoded_secret_or_debug`	T-06, T-07	no secret literal, no debug=True

A. Before: Pipeline Fails on the Vulnerable Baseline

PR #5 introduced the pipeline while the vulnerable code was on `main`. Both gating jobs failed and the Docker job was skipped (Fig. 32). ASAN reported a WRITE past the 32-byte buffer in `parse_header` (Fig. 33); six of eight exploit tests failed (Fig. 34); Bandit reported B602 (`subprocess` with `shell=True`, High, CWE-78, `app.py:214`) and B201 (Flask debug, High, CWE-94, `app.py:228`) (Fig. 35). This demonstrates the fail-on-critical policy.

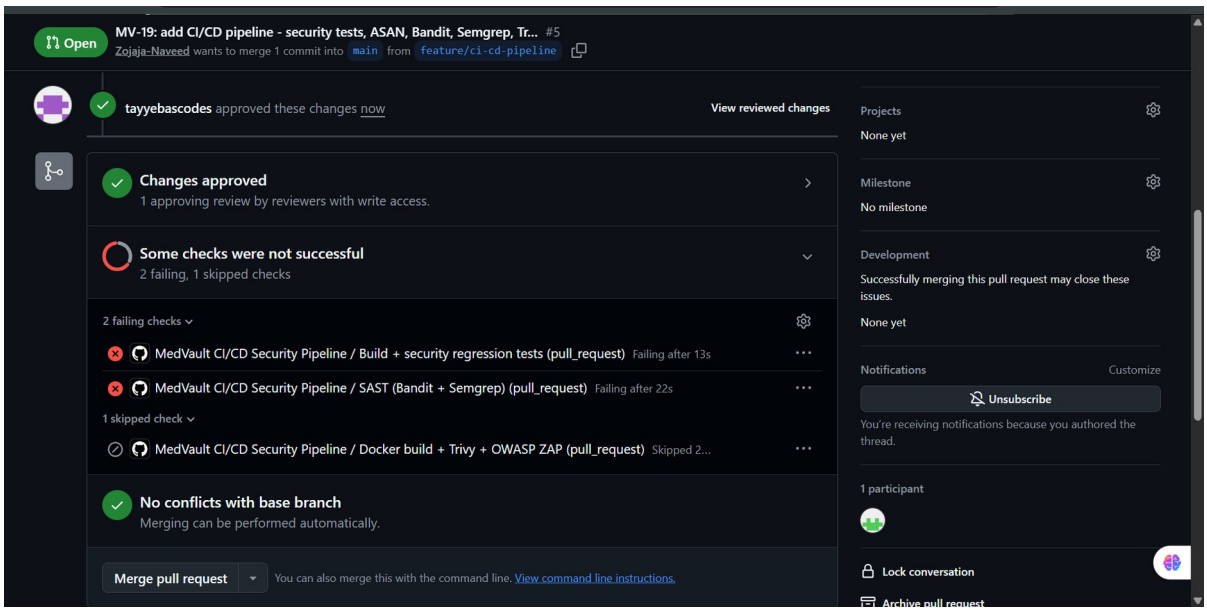


Fig. 32. PR #5: pipeline fails on the vulnerable code; Docker/Trivy/ZAP job skipped.

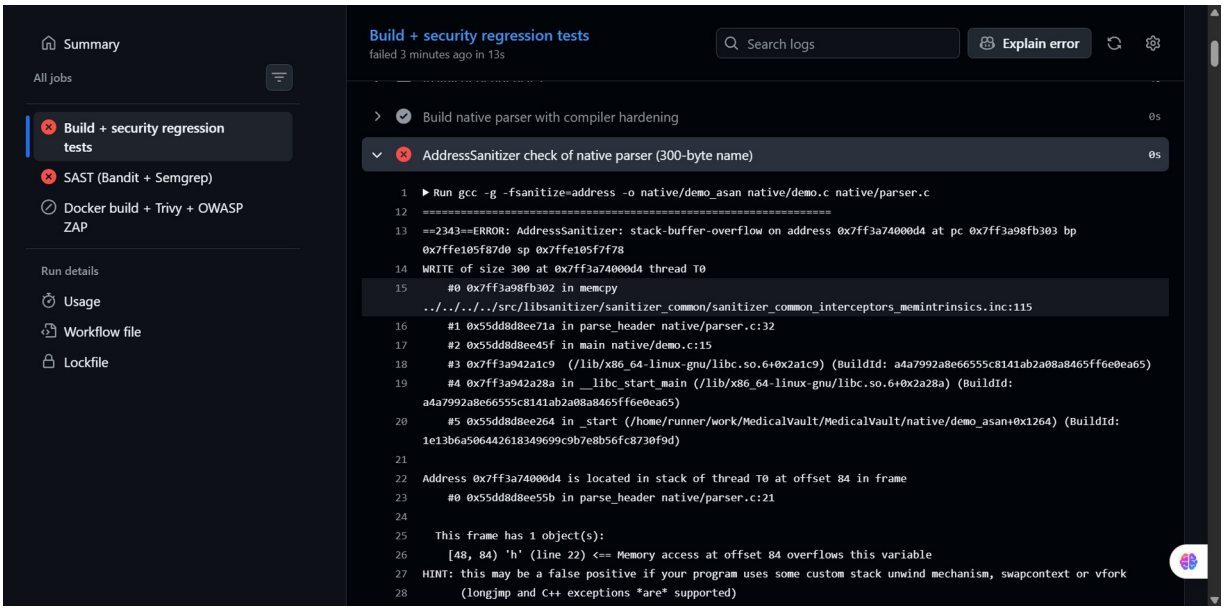


Fig. 33. AddressSanitizer: stack-buffer-overflow in parse_header.

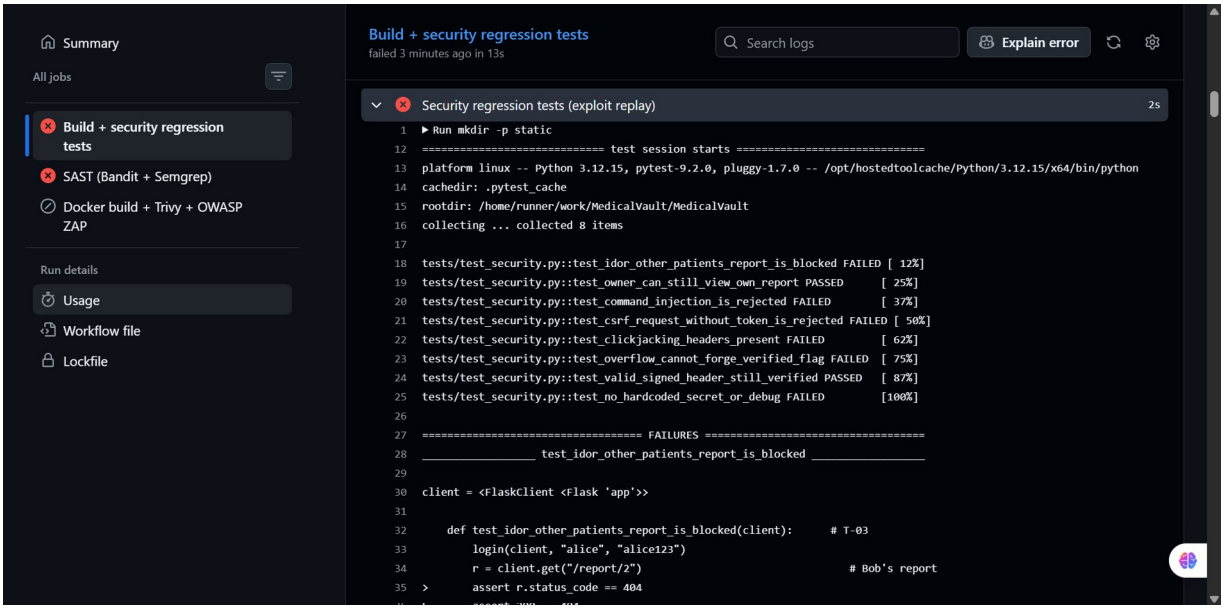


Fig. 34. Exploit-replay tests: 6 failed, 2 passed.

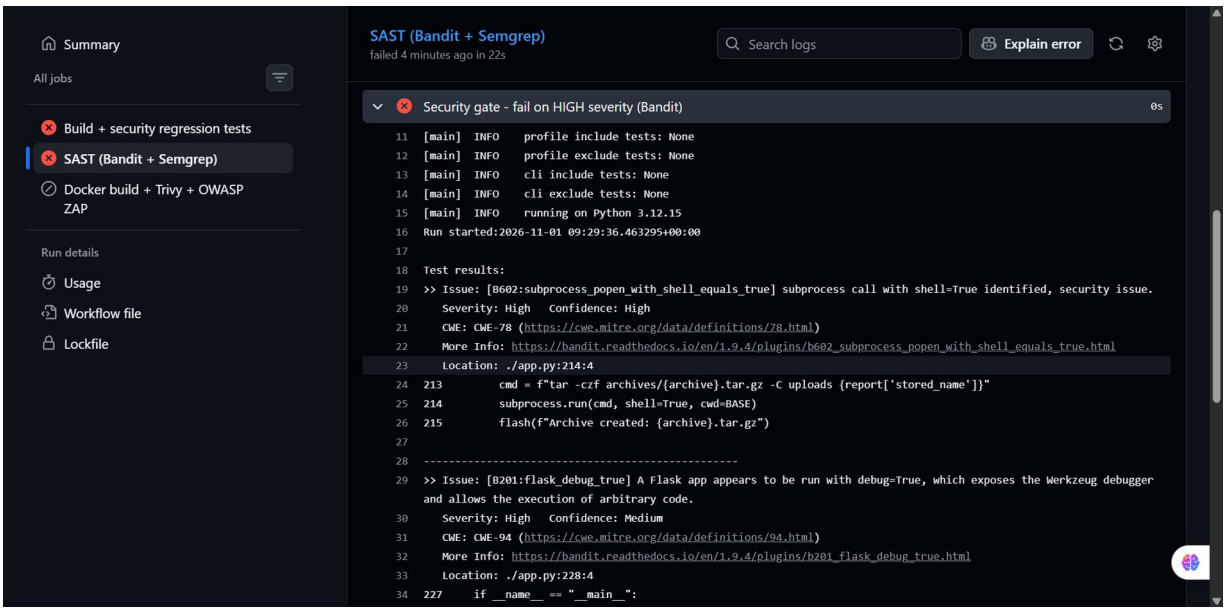


Fig. 35. Bandit security gate: two High-severity findings.

B. After: Pipeline Passes

PR #6 turned the ASAN step green while the web tests still failed, showing the gates are independent. PR #7 fixed the remaining classes and moved Trivy and ZAP to their official container images; all three jobs passed and the pull request became mergeable (Fig. 36).

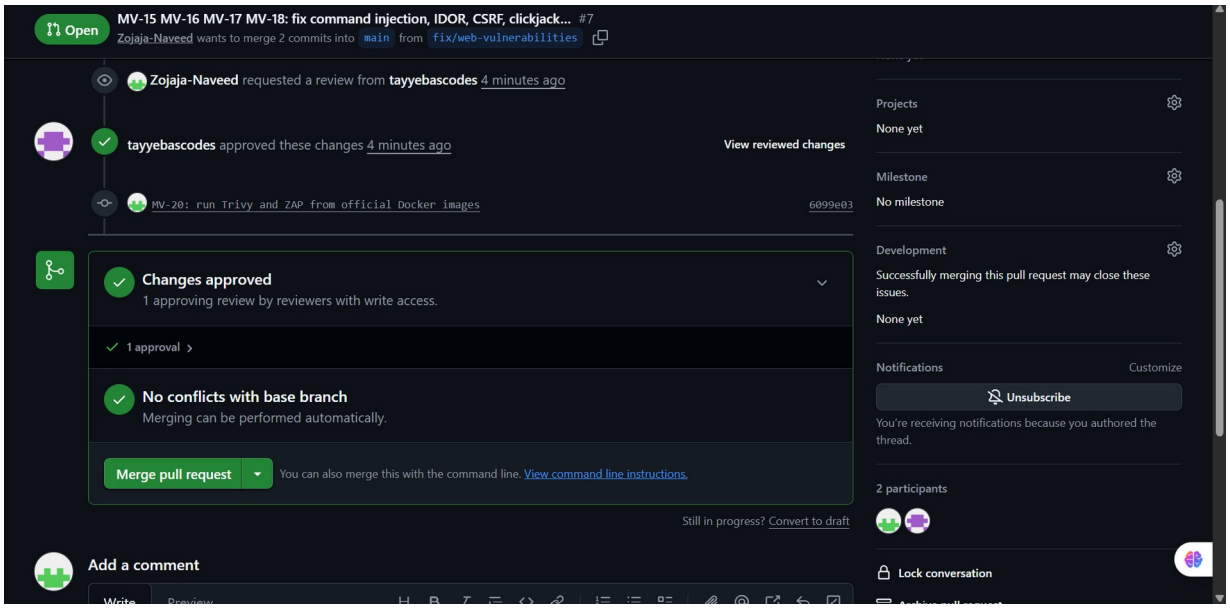


Fig. 36. PR #7: approved, checks passed and mergeable under the required-checks rule.

TABLE XI
SAST / DAST BEFORE AND AFTER

Check	Before (PR #5)	After (PR #7)
AddressSanitizer	stack-buffer-overflow	No error; name rejected
Exploit-replay tests	6 failed / 2 passed	8 passed
Bandit gate (High)	B602, B201	None (B603 argument-list call documented with nosec)
Semgrep	shell=True and debug patterns reported	Patterns removed
Trivy (Critical, fixable)	Not reached (skipped)	Passed
OWASP ZAP baseline	Not reached (skipped)	Completed; anti-framing and header alerts no longer raised

VIII. CIA Residual Risk Analysis

After remediation every register entry is rated Low or Very low. The following risks remain and are accepted for this sprint, each with a planned control.

TABLE XII
RESIDUAL RISKS BY CIA PROPERTY

Property	Residual risk	Likelihood / impact	Planned control
Confidentiality	No TLS in the local lab setup; cookies could be sniffed on an untrusted network	Medium / High	TLS reverse proxy; enable `MEDVAULT_SECURE_COOKIES=1`
Confidentiality	SQLite data and report files not encrypted at rest	Low / High	Volume or database encryption
Confidentiality, Integrity	No login rate limiting or MFA; weak lab passwords	Medium / Medium	Rate limiting (e.g., Flask-Limiter), MFA, password policy
Integrity	Lab signature is a static marker, not cryptographic (Attack Tree 2, branch 3)	Medium / High	HMAC or digital signature issued by the lab
Integrity, Availability	C parser is memory-unsafe by language	Low / Medium	Kept minimal behind length checks and hardening; candidate for a Rust rewrite; fuzzing
Availability	No request rate limiting on uploads/compression	Low / Medium	Rate limits and archive quotas

Overall, confidentiality depends most on the deployment layer (TLS, encryption at rest), integrity on replacing the static signature marker, and availability on rate limiting. None of these is reachable through the five mandatory vulnerability classes after the fixes.

IX. Lessons Learned

- **A gate that has never been red proves nothing.** Introducing the pipeline before the fixes gave hard evidence that it detects the vulnerabilities; the later green run then proved the fixes.
- **Exploits make the best regression tests.** Turning each manual proof-of-concept into a pytest case cost minutes and now prevents any fix from being silently reverted.
- **Defence in depth is cheap.** Allow-list plus `shell=False`, token plus `SameSite`, bounds check plus canary and FORTIFY: each pair means one mistake does not reopen the hole.
- **Centralize security decisions.** One `get_report_for_user` function and one `before_request` CSRF hook were easier to review and test than per-route checks.
- **Small memory corruption has large effects.** Four extra bytes changed a security decision without crashing anything — the most dangerous kind of bug because nothing looks wrong.
- **Pin to what exists.** A pipeline failed because an action version tag had been removed; running Trivy and ZAP from their official container images made the pipeline more robust.
- **Process evidence matters as much as code.** Jira keys in commits, PR reviews and screenshots taken at each step made the traceability chain easy to demonstrate.

X. Individual Contribution Statement

TABLE XIII
MEMBER CONTRIBUTIONS

Member	Registration No.	Contributions
Zojaja Naveed	FA23-BCT-047	Repository and branch-protection setup; Jira board, epics and sprint management; PNE report and threat model (DFDs, STRIDE, attack trees, CVSS, risk register); MedVault application and native parser; exploitation of all five classes and evidence capture; buffer-overflow and web-vulnerability fixes (PR #6, #7); CI/CD pipeline (PR #5); security documentation and final report.
Tayyeba Usman	FA23-BCT-053	Own commit via PR #2; peer review and approval of the pull requests, including the security fixes (#6, #7) and the pipeline (#5); merged PR #3 and PR #4; owner of assigned Jira tasks (repository setup, DFDs, CVSS and risk register, IDOR, clickjacking, Docker/Trivy/ZAP, SECURITY_IMPLEMENTATION).

Both members can explain every security-critical change. AI coding assistance was used, as permitted by the exam guidelines, to draft code and documentation; all changes were reviewed, run and verified by the team.

XI. Conclusion

MedVault was taken from a deliberately vulnerable baseline to a hardened release within one sprint. All five mandatory vulnerability classes were exploited in the lab and then fixed with standard, explainable controls: bounds checking with compiler hardening, shell-free process execution with an allow-list, centralized object-level authorization, synchronizer CSRF tokens with SameSite cookies, and anti-framing headers. Three additional findings were also fixed. The CI/CD pipeline failed on the baseline and passes on the fixed code, and branch protection makes those security gates mandatory for every future change. The remaining risks — transport and at-rest encryption, cryptographic lab signatures and rate limiting — are documented with planned controls.

References

[1] M. A. Nawaz, "CYC386 Secure Software Design and Development – Midterm Lab Exam: 1-Week DevSecOps Security Sprint," COMSATS University Islamabad, Fall 2026.

[2] OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/

[3] OWASP Foundation, "Cross-Site Request Forgery Prevention Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

[4] OWASP Foundation, "Clickjacking Defense Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Clickjacking_Defense_Cheat_Sheet.html

[5] OWASP Foundation, "OS Command Injection Defense Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html

[6] OWASP Foundation, "Authorization Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[7] MITRE, "CWE-120, CWE-78, CWE-639, CWE-352, CWE-1021," Common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/

[8] FIRST, "Common Vulnerability Scoring System v3.1: Specification Document," 2019. [Online]. Available: https://www.first.org/cvss/v3.1/specification-document

[9] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: Wiley, 2014.

[10] B. Schneier, "Attack trees," Dr. Dobb's Journal, vol. 24, no. 12, pp. 21–29, 1999.

[11] SEI CERT, "SEI CERT C Coding Standard," Carnegie Mellon University. [Online]. Available: https://wiki.sei.cmu.edu/confluence/display/c

[12] GitHub, "GitHub Actions documentation." [Online]. Available: https://docs.github.com/actions

[13] PyCQA, "Bandit documentation." [Online]. Available: https://bandit.readthedocs.io/

[14] Aqua Security, "Trivy documentation." [Online]. Available: https://trivy.dev/

[15] ZAP, "ZAP Baseline Scan." [Online]. Available: https://www.zaproxy.org/docs/docker/baseline-scan/

APPENDIX A. REPOSITORY STRUCTURE AND DELIVERABLES

TABLE undefined
DELIVERABLES AND THEIR LOCATION

Deliverable	Location in repository
Application source	`app.py`, `templates/`, `native/parser.c`, `Dockerfile`, `requirements.txt`
PNE report	`PNE_Report.pdf` (editable: `docs/PNE_Report.docx`)
Threat model	`THREAT_MODEL.pdf` (editable: `docs/THREAT_MODEL.docx`, diagrams in `docs/diagrams/`)
Security implementation	`SECURITY_IMPLEMENTATION.md`
Security test report	`SECURITY_TEST_REPORT.md`
CI/CD workflow	`.github/workflows/ci-cd.yml`, tests in `tests/test_security.py`
Repository hygiene	`README.md`, `SECURITY.md`, `CONTRIBUTING.md`
Evidence screenshots	`docs/screenshots/`
Final report	`Final_Report.pdf` (editable: `docs/Final_Report.docx`)
Lab exploit material	`samples/` (overflow payloads), `attacker/` (CSRF and clickjacking pages)
Vulnerable baseline	Commit `98478ce` (main before PR #6)

Repository: https://github.com/Zojaja-Naveed/MedicalVault